

Top-level statements en C#: ¿dónde quedó el resto del código?

Fundamentos de Programación (IAD-2413) — Instituto Tecnológico de Durango (TecNM)

El problema

Al crear un proyecto de consola en C# con:

```
dotnet new console -o HolaMundo
```

VS Code muestra un Program.cs que contiene **solo esta línea**:

```
Console.WriteLine("Hola, Mundo");
```

Pero en la Unidad 1 vimos que un programa en C# necesita una clase y un método Main:

```
using System;

class Programa
{
    static void Main()
    {
        Console.WriteLine("Hola, Mundo");
    }
}
```

¿Dónde quedó todo lo demás?

La explicación: top-level statements

Desde **.NET 6**, Microsoft introdujo una característica llamada **top-level statements** (instrucciones de nivel superior). Permite escribir el código ejecutable directamente en la raíz del archivo, sin declarar la clase ni el método Main de forma explícita.

Cuando **compilas** el programa, el compilador genera automáticamente, por detrás, una estructura equivalente a la que ya conoces:

```
class Program
{
    static void <Main>$(string[] args)    // nombre interno del compilador
    {
```

```

        Console.WriteLine("Hola, Mundo");
    }
}

```

La clase y el método de entrada **siguen existiendo**; el compilador solo te ahorra escribirlos. El método generado se llama <Main>\$, un nombre interno que no puedes invocar desde tu código.

El `using System;` tampoco aparece en `Program.cs` porque los proyectos nuevos activan **ImplicitUsings** en el `.csproj` (`<ImplicitUsings>enable</ImplicitUsings>`), que agrega `using System;` y otros espacios de nombres de forma automática. Si ese ajuste se desactivara, habría que escribir el `using` a mano.

Punto clave

Ambas versiones son válidas en C# y **compilan al mismo resultado**. La diferencia es únicamente de qué tanto código “boilerplate” (repetitivo) tienes que escribir tú mismo.

	Top-level statements	Versión explícita
¿Qué escribes?	Solo el código ejecutable	Clase + método Main + código
¿Qué genera el compilador?	La clase y el Main implícitos	Nada adicional, tú ya lo escribiste todo
¿Es válido en C#?	Sí	Sí
¿Compilan igual?	Sí, son equivalentes	Sí, son equivalentes
Uso recomendado	Programas pequeños, prototipos, scripts	Cuando quieres que la estructura sea explícita (por ejemplo, para fines didácticos)

¿Puedo escribir ambas formas en el mismo archivo?

Depende de cómo lo hagas:

☒ Reemplazar la línea por el código completo — sin error

Si **quitas** `Console.WriteLine(...)` suelto y en su lugar escribes solamente la clase con su Main, el programa compila y funciona igual:

```

using System;

class Programa
{
    static void Main()

```

```

    {
        Console.WriteLine("Hola, Mundo");
    }
}

```

⚠ Dejar ambas formas en el mismo archivo — compila, pero tu Main se ignora

Si dejas la línea suelta **y además** agregas una clase con su propio Main, el compilador **no marca error**: emite una **advertencia** y ejecuta únicamente la línea suelta.

```
using System;
```

```
Console.WriteLine("Hola desde la línea suelta");
```

```

class Programa
{
    static void Main()
    {
        Console.WriteLine("Hola desde Main");    // nunca se ejecuta
    }
}

```

warning CS7022: The entry point of the program is global code; ignoring 'Programa.Main()' entry point.

Salida del programa: Hola desde la línea suelta. El Main de la clase queda ignorado, lo cual suele confundir a quien lo escribió pensando que se ejecutaría.

Reglas que sí producen error (verificadas con el compilador):

Situación	Resultado
Línea suelta antes de los using (la línea suelta va antes de using System;)	error CS1529: una cláusula using debe preceder a todos los demás elementos del espacio de nombres
Dos archivos .cs del proyecto, ambos con instrucciones sueltas	error CS8802: solo una unidad de compilación puede tener instrucciones de nivel superior
Dos métodos Main en el proyecto, sin instrucciones sueltas	error CS0017: el programa tiene más de un punto de entrada

Recomendación didáctica: elige **una sola forma** por proyecto. Mezclarlas no rompe la compilación, pero deja código que nunca se ejecuta.

Cómo obtener siempre la versión explícita al crear un proyecto

No existe una propiedad del archivo `.csproj` que “desactive” los top-level statements después de crear el proyecto — no es un interruptor del compilador, sino una forma distinta de escribir el mismo código. El ajuste se aplica **al momento de crear el proyecto**, mediante una bandera del comando `dotnet new`:

```
dotnet new console -o HolaMundo --use-program-main
```

Esta bandera está disponible desde el SDK de .NET 6.0.300 en adelante y genera directamente el `Program.cs` con la clase y el `Main` explícitos — la misma estructura que verán en las diapositivas del curso.

Si el proyecto ya fue creado sin esta bandera, **no es necesario recrearlo**: basta con reemplazar el contenido de `Program.cs` por la versión explícita a mano, como se mostró arriba.

Resumen para recordar

1. Los **top-level statements** son un atajo del compilador, no una forma distinta de ejecutar un programa: ambas versiones compilan a lo mismo.
2. **No mezcles** una instrucción suelta con una clase que define `Main` en el mismo archivo: compila con la advertencia CS7022 y solo se ejecuta la línea suelta; tu `Main` se ignora.
3. Para que `dotnet new console` genere siempre la versión explícita, usa la bandera `--use-program-main` al crear el proyecto.
4. No hay ninguna propiedad del `.csproj` que resuelva esto; es una decisión que se toma al generar el archivo, no una configuración del proyecto ya creado.

Material de apoyo — Departamento de Sistemas y Computación, Instituto Tecnológico de Durango (TecNM)